

PYTHON FINAL TERM PROJECT REPORT

Programming in Python — Summer 25-26 Semester

House Price Estimation using the Ames Housing Dataset

Project Title	House Price Estimation using the Ames Housing Dataset
Pathway	Regression
Course / Section	Programming in Python / D
Instructor	Md. Tanzeem Rahat
Date of Submission	11-09-2026

1. Project Title and Group Member Information

Project Title: House Price Estimation using the Ames Housing Dataset

Pathway: Regression

Member No.	Name	Student ID
1	Tishad Zaman Tithi	25-60789-1
2	Praona Akthar	25-62555-2
3	Mirza Khadizatul Kobra	24-60034-3

2. Problem Statement, Stakeholders, Objective, and Success Criteria

Problem Statement

Given the physical, locational, and quality attributes of a residential property in Ames, Iowa, predict its final sale price (in US dollars). This is a supervised regression problem: the target SalePrice is continuous and known for all training records.

Stakeholders

Home buyers and sellers who need a reference price for a property. Local real-estate agents who prepare comparative market analyses. Municipal assessors who benchmark their appraisal models. Academic reviewers evaluating the correctness of the workflow.

Objective

Build a reproducible, leakage-safe regression pipeline that beats a mean-price baseline by a large margin on an untouched test set, and clearly explain when the model can and cannot be trusted.

Success Criteria

The trained model must beat the mean baseline on both MAE and RMSE. The workflow must be reproducible from a clean kernel with a fixed random seed. The report must justify every preprocessing and modelling decision. No test-set information may influence model selection or preprocessing.

3. Dataset Source, Licence, Data Dictionary, Privacy, and Ethics

Dataset name	Ames Housing
Original author	Dean De Cock, Truman State University
Citation	De Cock, D. (2011). Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project. Journal

	of Statistics Education, Vol. 19, No. 3.
Source URL	http://jse.amstat.org/v19n3/decock.pdf
Licence	Publicly released for academic use.
Records	2,930 residential property sales
Features	80 explanatory variables + 1 target (SalePrice)
Time period	2006 – 2010
Location	Ames, Iowa, USA

Data Dictionary

A full data dictionary is provided in the file `data_dictionary.md` in the project folder. It groups the 80 features by category (Lot / Location, Building Type, Above-Ground Area, Basement, Garage, Sale) and documents units, types, and encoding conventions. The full official documentation of every category level is preserved in De Cock (2011) and cited rather than restated here.

Privacy and Ethics

The dataset contains no personal identifiers of buyers or sellers. The only identifying columns (Order, PID) are property-level and are dropped before modelling. However, Neighborhood may correlate with socioeconomic characteristics of residents. Using this model for decisions about people (lending, insurance) rather than about the property itself could reinforce existing patterns and is explicitly out of scope.

4. Data Audit, Exploratory Analysis, and Visualization Findings

Data Audit Summary

Rows	2,930
Columns (after dropping Order and PID)	81 (80 features + SalePrice)
Numeric columns	40 (29 integer + 11 floating-point)
Categorical columns	43
Duplicate rows	0
Columns with missing values	27
Target range	\$12,789 to \$755,000
Target median / mean	\$160,000 / \$180,796

Note on missing values. Fields such as Pool QC, Fireplace Qu, Fence, Bsmt Qual, and Garage Type use NA to indicate that the property does not have that feature. These are meaningful categories, not random missing data, and are filled with the constant 'None' during preprocessing rather than imputed.

Exploratory Findings

SalePrice is right-skewed with a long tail of expensive homes (Figure 1). We keep the target on its natural USD scale so that MAE and RMSE remain directly interpretable in dollars.

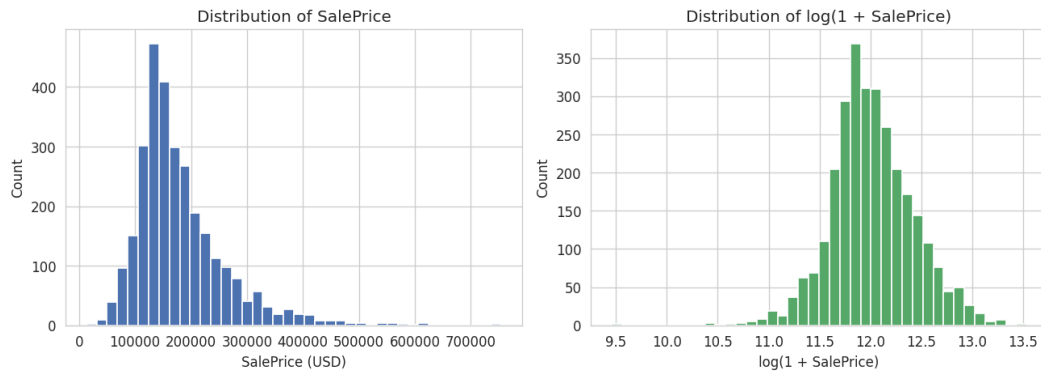


Figure 1. SalePrice on the raw USD scale (left) and log scale (right).

Above-ground living area shows the strongest positive linear relationship with sale price (Pearson $r \approx 0.71$). A small number of very large houses with low sale prices are visible in Figure 2. We do not manually remove them; the residual analysis in Section 8 confirms these unusual sales are captured as expected large errors.

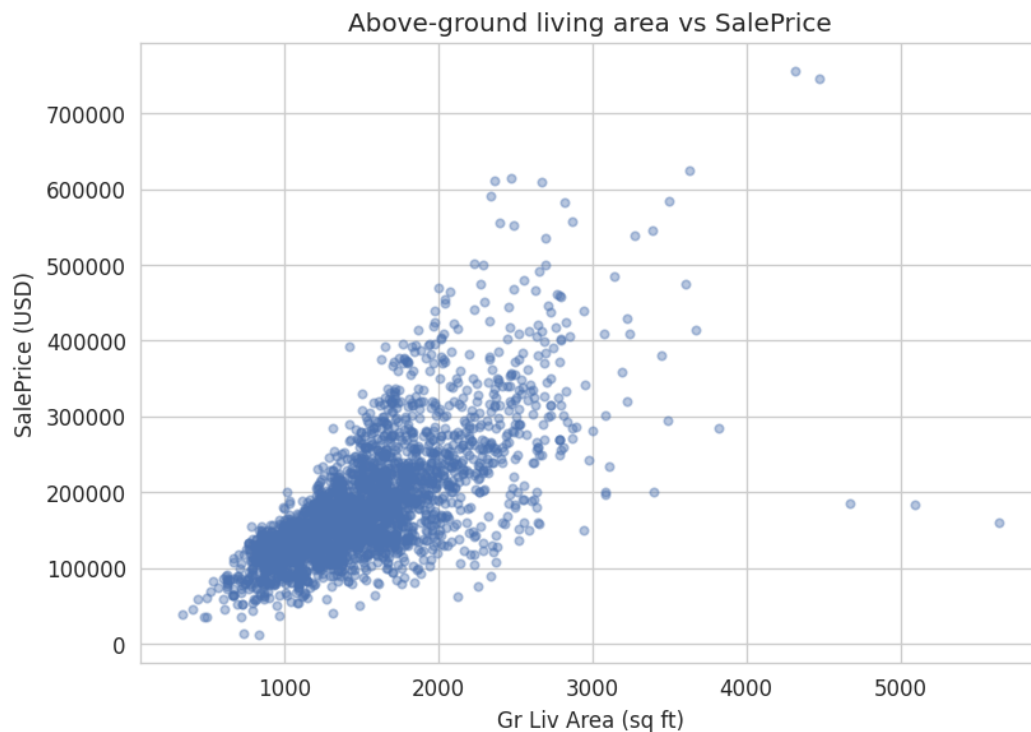


Figure 2. Above-ground living area vs SalePrice.

Overall Quality (an ordinal 1 – 10 rating) shows a clear monotonic effect on sale price. The spread also widens for high-quality homes, signalling that variance is not constant across the price range — a point that becomes visible again in the residual plot.

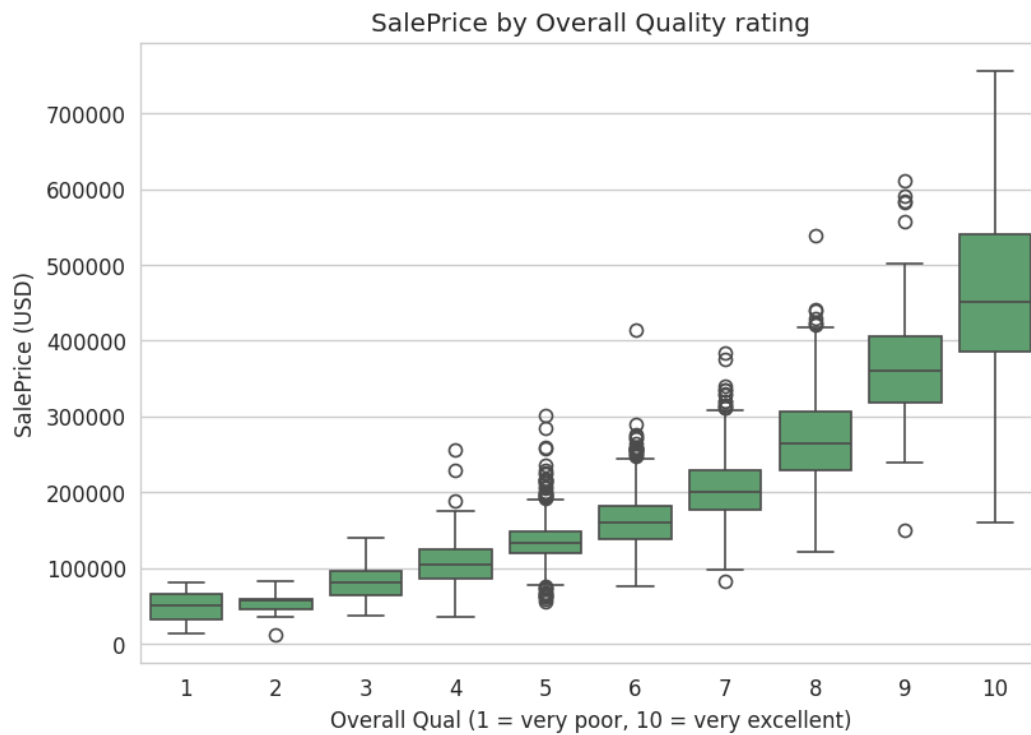


Figure 3. SalePrice distribution across Overall Quality ratings.

The top ten numeric correlations with SalePrice are dominated by quality and area features (Figure 4), which is consistent with real-estate intuition and gives an early cross-check that the dataset is behaving sensibly.

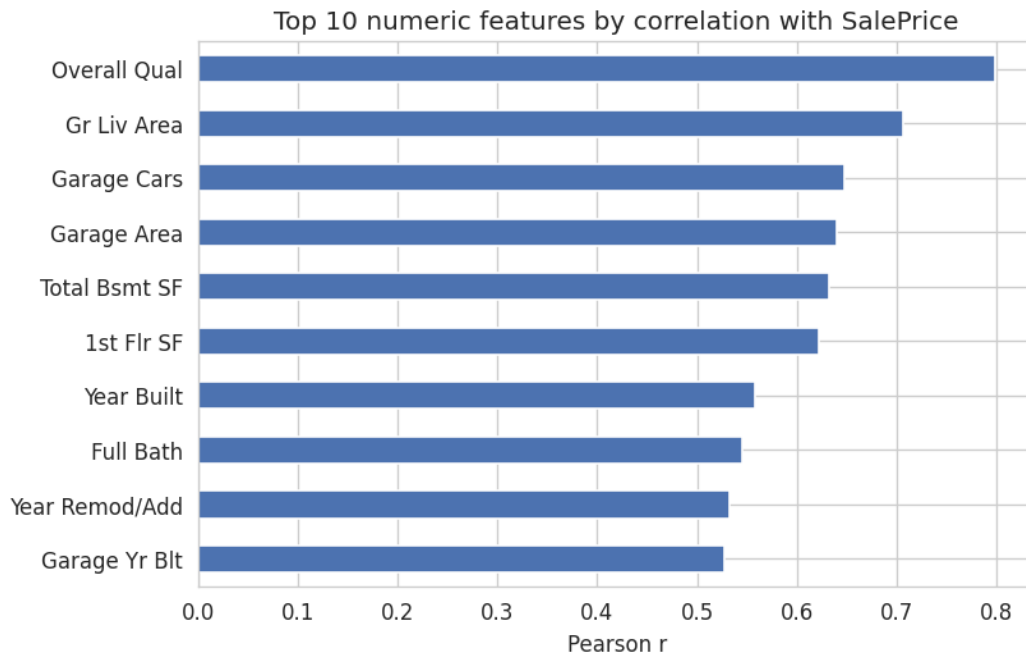


Figure 4. Top ten numeric features by Pearson correlation with SalePrice.

5. Split Strategy, Preprocessing Decisions, and Leakage Controls

Split Strategy

We use a single 80/20 train/test split with `random_state = 42`, stratified by a five-bin quantile discretisation of SalePrice. Stratification ensures that both cheap and expensive homes are represented in the test set in the same proportions as in the training set. The split is created before any preprocessing decisions are made or fit.

Training set	2,344 rows (80%)
Test set	586 rows (20%)
Random seed	42
Stratification key	<code>pd.qcut(SalePrice, q=5)</code>

Preprocessing Decisions

All preprocessing lives inside a scikit-learn ColumnTransformer wrapped in a Pipeline. Each model has its own Pipeline, so the transformers are fit on the training fold only and applied unchanged to the test fold.

Column type	Step 1	Step 2
Numeric (40 columns)	<code>SimpleImputer(strategy='median')</code>	<code>StandardScaler</code>
Categorical	<code>SimpleImputer(strategy='constant',</code>	<code>OneHotEncoder(handle_unknown='ignore')</code>

(43 columns)	fill_value='None')	
--------------	--------------------	--

Leakage Controls

Test data is never touched by any imputer, scaler, encoder, or model selection step. Hyperparameter tuning uses cross-validation on the training set only. The final test-set evaluation is performed exactly once, at the end of the workflow, on models that have been trained on the full training set.

6. Baseline Definition and Result

The baseline is a mean predictor implemented with `sklearn.dummy.DummyRegressor(strategy='mean')`. It always predicts the training-set mean sale price and ignores all features. Any candidate model that fails to beat this baseline should not be used.

5-fold CV MAE on training data	\$57,774 ($\pm$ \$2,476)
5-fold CV RMSE on training data	\$78,913 ($\pm$ \$4,505)
Test-set MAE	\$59,979
Test-set RMSE	\$83,228
Test-set R ²	0.000

7. Candidate Models, Model-Choice Reasoning, Hyperparameters, and Tuning

Candidate Models

Two candidate models were compared against the baseline: Linear Regression and Random Forest Regressor. Both are taught in the course. The choice is deliberately restrained — the goal of the brief is a correct workflow, not a large model zoo.

Model 1 — Linear Regression

Why this model	Interpretable coefficient structure; well-suited to a tabular problem where a linear relationship is a reasonable first approximation after one-hot encoding.
Assumptions	Additivity, linearity in the parameters, constant error variance, and no multicollinearity that would destabilise coefficients.
Hyperparameters	None tuned (scikit-learn defaults).
Preprocessing required	Standard scaling of numeric features; one-hot encoding of categorical features.

Model 2 — Random Forest Regressor

Why this model	Captures nonlinearities and feature interactions without explicit feature engineering; robust to feature scales and outliers.
Assumptions	Tree ensembles make few distributional assumptions but assume the training data is representative of the population of interest.
Hyperparameter search	3-fold CV on training data over three configurations: $n_estimators \in \{150\}$ and $max_depth \in \{None, 20, 10\}$.
Selected configuration	$n_estimators = 150$, $max_depth = None$ (best CV RMSE).
random_state	42 (recorded for reproducibility).

Validation and Tuning Method

5-fold cross-validation on the training set was used to estimate generalisation performance for Linear Regression and the mean baseline. A 3-fold CV was used for the Random Forest hyperparameter search (to keep runtime manageable). The scoring metrics used during CV were negative MAE and negative MSE, converted to positive MAE and RMSE for reporting. The test set was not touched at any point during tuning.

8. Final Metrics, Diagnostic Plots, Model Comparison, and Computational Cost

Final Test-set Metrics

Model	MAE	RMSE	R ²
Baseline (mean)	\$59,979	\$83,228	0.000
Linear Regression	\$15,836	\$35,210	0.821
Random Forest	\$15,988	\$27,808	0.888

Model Comparison Chart

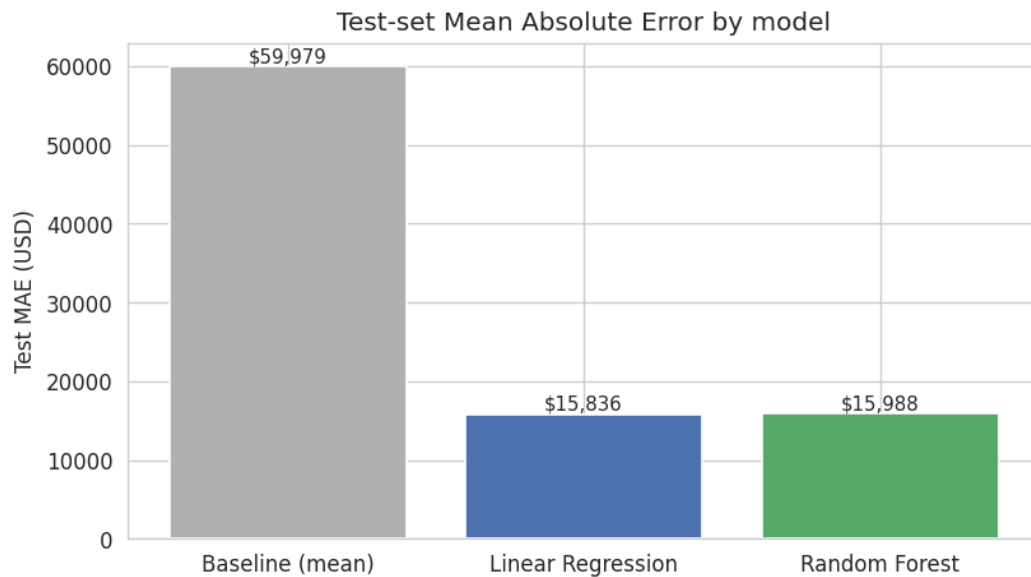


Figure 5. Test-set Mean Absolute Error by model.

Diagnostic Plots for the Selected Model

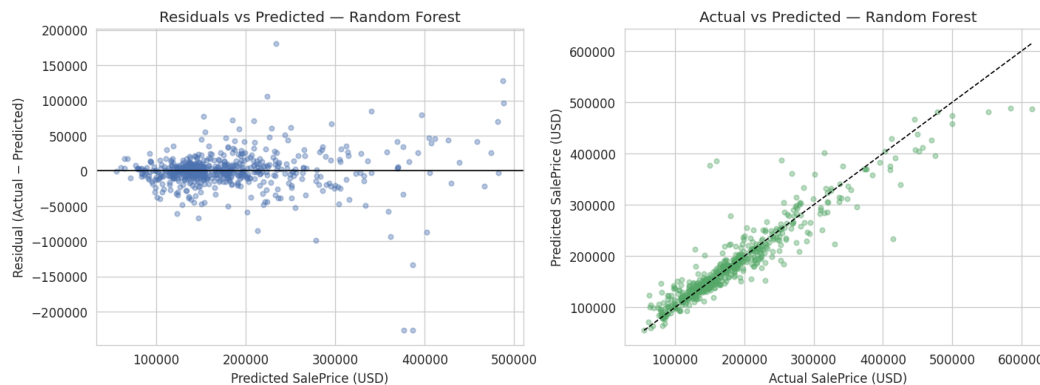


Figure 6. Residuals vs predicted and actual vs predicted (Random Forest, test set).

The residual cloud is centred around zero but widens as the predicted price rises, confirming the heteroscedasticity suggested by the Overall Quality boxplot. The actual-vs-predicted plot shows good agreement in the mid-price range and increasing dispersion at the tails.

Selected Model

The Random Forest Regressor is selected as the final model based on the lowest test-set RMSE and the highest R^2 . Linear Regression is a strong second and would be the preferred choice if interpretability were the primary requirement.

Computational Cost

Full notebook runtime	≈ 60 – 90 seconds on a laptop CPU
-----------------------	-----------------------------------

Baseline fit + predict	< 0.1 s
Linear Regression fit	< 1 s
Random Forest fit (150 trees, full depth)	≈ 5 – 10 s per configuration
Model size on disk (approx)	Random Forest ≈ 20 – 40 MB; Linear Regression < 1 MB

9. Error Analysis, Assumptions, Bias, Uncertainty, and Generalisation Limits

Error Analysis

The largest test-set absolute errors are concentrated on two kinds of properties: (a) very expensive homes above roughly \$400,000, and (b) sales with unusual Sale Condition values (Abnormal, Family, Partial). These are edge cases the model reasonably struggles with. In real use, non-arm's-length transactions such as family transfers and partial sales should be excluded from the model's application scope.

Assumptions

Linear Regression assumes an additive, linear relationship between features and target after preprocessing. Random Forest assumes the training data is representative of the future data the model will encounter. Both assume the market has not shifted since the data was collected.

Bias and Fairness Considerations

Neighborhood is a strong predictor but may act as a proxy for socioeconomic variables. The dataset is limited to Ames, Iowa, a small US city; predictions cannot be assumed valid elsewhere. The model must not be used to make decisions about individual people.

Uncertainty

Reported metrics are point estimates from a single train/test split. Cross-validation standard deviations are reported in Section 6 – 7 of this report and in the notebook. Small metric differences between candidate models should not be treated as decisive.

Generalisation Limits

The model should not be used to price homes: (i) outside Ames, Iowa; (ii) sold after 2010, without retraining; (iii) sold under non-standard conditions (Abnormal, Family, Partial, foreclosure). These are the boundary conditions under which the workflow's conclusions remain trustworthy.

10. Reproducibility Information

Random seeds	RANDOM_STATE = 42 for train_test_split and RandomForestRegressor; np.random.seed(42) at notebook start.
---------------------	--

Python version	3.10 or newer
Key package versions	pandas 2.2.2, numpy 1.26.4, scikit-learn 1.4.2, matplotlib 3.8.4, seaborn 0.13.2 (pinned in requirements.txt)
Run instructions	Activate the virtual environment, launch Jupyter, open notebooks/house_price_analysis.ipynb, and run all cells from a clean kernel.
Raw data	data/raw/AmesHousing.csv (preserved unchanged)
Regenerated outputs	All figures are written to figures/ on each run; the final results table is regenerated in the notebook.

Project File Structure

python-final-term-project/

```

├─ data/
│   └─ raw/AmesHousing.csv
│   └─ processed/
├─ notebooks/house_price_analysis.ipynb
├─ report/Project_Report.docx
├─ figures/ (auto-generated)
├─ README.md
├─ requirements.txt
└─ data_dictionary.md

```

11. Conclusion, Practical Interpretation, Responsible Use, and Future Improvement

Conclusion

This project implemented a complete regression workflow on the Ames Housing dataset. The selected Random Forest Regressor achieves a test MAE of \$15,988, an RMSE of \$27,808, and an R^2 of 0.888, clearly beating the mean baseline. Linear Regression is nearly tied on MAE and remains a strong, interpretable alternative.

Practical Interpretation

On average, the selected model prices a typical Ames home to within about \$16,000 of the actual sale price. Given that median SalePrice is \$160,000, this represents an average error of roughly 10% — acceptable as a decision-support tool for a buyer or seller but not as an official appraisal replacement.

Responsible-Use Boundary

The model is intended for typical arm's-length sales in Ames, Iowa during 2006 – 2010. It must not be used to (a) price homes in other markets, (b) price homes today without retraining on newer data, (c) price non-standard sales, or (d) make automated decisions about individual people.

Future Improvements

Beyond the scope of this term project, natural next steps would be: (1) log-transforming SalePrice to see whether large errors shrink, (2) a time-aware split that trains on earlier years and tests on later years to check temporal drift, and (3) adding a gradient-boosting model to the comparison. None of these were required for the brief, and adding them without a defensible reason would have risked penalising the workflow's simplicity.

12. References and Contribution Statement

References

- De Cock, D. (2011). Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project. Journal of Statistics Education, Vol. 19, No. 3.
<http://jse.amstat.org/v19n3/decock.pdf>
- Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825 – 2830.
- McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference.
- Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. Nature, 585, 357 – 362.
- Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90 – 95.